Práctica 1: Complejidad Algorítmica

Cálculo del número hexagonal n

Para calcular el número hexagonal de tamaño n tenemos 3 opciones.

Opción 1: Fórmula.

La primera opción es aplicar la fórmula$H_n = 2 \cdot n^2 - n$. Como podemos observar la complejidad de este algoritmo es de O(n^2), ya que es una sola operación en la que se ve involucrado un polinomio de grado 2.

Opción 2: Algoritmo iterativo.

La segunda opción consiste en aplicar la fórmula $H_n = \sum_{i=0}^{n-1}(4i + 1)$de forma iterativa. La implementación de dicho algoritmo en java se materializa en el método *calculoSerieIterativo*. En el fichero NumeroHexagonal.java analizamos la complejidad de este algoritmo, resultando en O(n).

Opción 3: Algoritmo recursivo.

La tercera opción consiste en aplicar la anterior fórmula de forma recursiva. La implementación de este algoritmo se puede ver en el método *calculoSerieRecursivo*. En el fichero NumeroHexagonal.java analizamos igualmente la complejidad, resultando en O(n).

A continuación, vamos a ejecutar los distintos algoritmos y estudiaremos los resultados obtenidos para sacar las apropiadas conclusiones.

Resultados obtenidos

Computador 1 (Óscar Javier Castillo García)

ESPECIFICACIONES:

- CPU: AMD Ryzen 7 4800H with Radeon Graphics @ 16x 4300,9MHz
- GPU: NVIDIA GeForce GTX 1650
- RAM: 16GB

```
Cómo desea ver los tiempos de ejecución? (nano/mili): nano

Tabla de números hexagonales (calculado con una media de 30 ejecuciones) :
-------------------------------------------------------------------------------
| N     | Valor      | Fórmula         | Iterativo       | Recursivo       |
-------------------------------------------------------------------------------
| 10    | 190        | 102,00       ns | 270,00       ns | 1693,00      ns |
| 100   | 19900      | 72,00        ns | 1177,00      ns | 710,00       ns |
| 500   | 499500     | 80,00        ns | 5730,00      ns | 1425,00      ns |
| 1000  | 1999000    | 78,00        ns | 10660,00     ns | 3327,00      ns |
| 5000  | 49995000   | 96,00        ns | 9551,00      ns | 16685,00     ns |
| 8000  | 127992000  | 72,00        ns | 19766,00     ns | 23741,00     ns |
| 10000 | 199990000  | 91,00        ns | 2799,00      ns | 37923,00     ns |
| 11000 | 241989000  | 110,00       ns | 3000,00      ns | 40208,00     ns |
| 12000 | 287988000  | 321,00       ns | 3240,00      ns | 16777,00     ns |
oscar@oscar-ROG-Strix-G513IH-G513IH:~/Escritorio/uni/2do año/2doCuatri/Metodol
```

```
Cómo desea ver los tiempos de ejecución? (nano/mili): mili

Tabla de números hexagonales (calculado con una media de 30 ejecuciones) :
-------------------------------------------------------------------------------
| N     | Valor      | Fórmula         | Iterativo       | Recursivo       |
-------------------------------------------------------------------------------
| 10    | 190        | 0,000117     ms | 0,000302     ms | 0,001769     ms |
| 100   | 19900      | 0,000119     ms | 0,001733     ms | 0,000851     ms |
| 500   | 499500     | 0,000104     ms | 0,008547     ms | 0,002024     ms |
| 1000  | 1999000    | 0,000112     ms | 0,013498     ms | 0,005750     ms |
| 5000  | 49995000   | 0,000175     ms | 0,013593     ms | 0,024524     ms |
| 8000  | 127992000  | 0,000100     ms | 0,031926     ms | 0,039540     ms |
| 10000 | 199990000  | 0,000092     ms | 0,003062     ms | 0,039439     ms |
| 11000 | 241989000  | 0,000083     ms | 0,002878     ms | 0,040695     ms |
| 12000 | 287988000  | 0,000158     ms | 0,003153     ms | 0,024724     ms |
oscar@oscar-ROG-Strix-G513IH-G513IH:~/Escritorio/uni/2do año/2doCuatri/Metodo
```

Computador 2 (Héctor Espinosa Navarro)

ESPECIFICACIONES:

- CPU: AMD Ryzen 5 PRO 2500U
- GPU: AMD Radeon Vega 8 Graphics
- 16 GB de memoria RAM

```
Cómo desea ver los tiempos de ejecución? (nano/mili): nano

Tabla de números hexagonales (calculado con una media de 30 ejecuciones) :
-------------------------------------------------------------------------------
| N     | Valor      | Fórmula         | Iterativo       | Recursivo       |
-------------------------------------------------------------------------------
| 10    | 190        | 258,00       ns | 666,00       ns | 2614,00      ns |
| 100   | 19900      | 228,00       ns | 4108,00      ns | 2077,00      ns |
| 500   | 499500     | 225,00       ns | 16891,00     ns | 4727,00      ns |
| 1000  | 1999000    | 246,00       ns | 21982,00     ns | 9668,00      ns |
| 5000  | 49995000   | 347,00       ns | 26387,00     ns | 46389,00     ns |
| 8000  | 127992000  | 253,00       ns | 56775,00     ns | 69469,00     ns |
| 10000 | 199990000  | 260,00       ns | 6964,00      ns | 89226,00     ns |
| 11000 | 241989000  | 214,00       ns | 8817,00      ns | 90959,00     ns |
| 12000 | 287988000  | 510,00       ns | 8222,00      ns | 53501,00     ns |
```

```
Cómo desea ver los tiempos de ejecución? (nano/mili): mili

Tabla de números hexagonales (calculado con una media de 30 ejecuciones) :
-------------------------------------------------------------------------------
| N     | Valor      | Fórmula         | Iterativo       | Recursivo       |
-------------------------------------------------------------------------------
| 10    | 190        | 0,000830     ms | 0,000688     ms | 0,002671     ms |
| 100   | 19900      | 0,000246     ms | 0,004055     ms | 0,001421     ms |
| 500   | 499500     | 0,000240     ms | 0,020291     ms | 0,004109     ms |
| 1000  | 1999000    | 0,000235     ms | 0,025441     ms | 0,010486     ms |
| 5000  | 49995000   | 0,000285     ms | 0,027711     ms | 0,051261     ms |
| 8000  | 127992000  | 0,000214     ms | 0,066504     ms | 0,059591     ms |
| 10000 | 199990000  | 0,000213     ms | 0,007085     ms | 0,090674     ms |
| 11000 | 241989000  | 0,000302     ms | 0,009644     ms | 0,114489     ms |
| 12000 | 287988000  | 0,000568     ms | 0,009354     ms | 0,041107     ms |
```

Conclusiones

Como podemos observar, los tiempos para el algoritmo 1 (fórmula) crecen por lo general menos rápidamente que los algoritmos iterativo y recursivo. Esto puede deberse a diversos factores externos como optimizaciones del compilador de la JVM o el funcionamiento interno de java (principalmente en el caso de las llamadas recursivas).

Aún así, podemos seguir afirmando que en la teoría, los algoritmos iterativo y recursivo son mucho más óptimos y eficientes que la fórmula.